

Day 01: Python History, Philosophy & Capabilities

Welcome to your first day of Python programming! Before we write any code, it is essential to understand where Python came from, why it was designed the way it was, and what makes it such a powerful tool in modern computing (especially in Artificial Intelligence and Data Science).

1. A Brief History of Python

Python was conceived in the **late 1980s** by **Guido van Rossum** at the *Centrum Wiskunde & Informatica* (CWI) in the Netherlands.

- Implementation of the language began in **December 1989** as a hobby project. Guido wanted a successor to the **ABC programming language** that could interface with the **Amoeba distributed operating system** and handle exceptions.
- **First Release (0.9.0)**: February 1991. It included classes, inheritance, exception handling, functions, and core data types like lists, dicts, and strings.
- **Python 2.0**: Released in October 2000. It introduced list comprehensions, garbage collection, and Unicode support.
- **Python 3.0**: Released in December 2008. It was a major, backward-incompatible release designed to fix structural design flaws in the language (such as fixing string representation to be Unicode by default and streamlining division).

[!NOTE] Guido van Rossum was known as Python's **Benevolent Dictator for Life (BDFL)** until he stepped down from the role in July 2018. The language is now governed by a five-member steering committee.

Why the name "Python"?

Contrary to popular belief, Python was not named after the snake. Guido van Rossum named the language after the British comedy group **Monty Python**, as he was reading published scripts from *"Monty Python's Flying Circus"* at the time and wanted a name that was short, unique, and slightly mysterious.

2. The Intent Behind Python (Design Philosophy)

Guido's goal was to design a language that was easy to read, write, and maintain. The core philosophy of Python is summarized in **The Zen of Python** (written by software engineer Tim Peters). You can read it in any Python console by typing:

```
import this
```

Key design tenets include:

- **Readability:** Code is read much more often than it is written. Python uses clean English-like keywords and relies on formatting indentation rather than braces or semicolons.
 - **Developer Time over CPU Time:** Computers are cheap, but developer time is expensive. Python focuses on rapid prototyping and clear syntax.
 - **Explicit over Implicit:** Code should not make magic assumptions.
 - **One Clear Way:** There should be one—and preferably only one—obvious way to solve a problem.
-

3. What are Python's Capabilities?

Python is a **general-purpose, high-level, interpreted, dynamically typed** programming language. Its versatility enables it to power systems across diverse domains:

Key Core Strengths

1. **Multi-Paradigm Support:** You can write code using **Procedural**, **Object-Oriented (OOP)**, or **Functional** programming styles.
2. **Dynamically Typed:** Variable types are determined at runtime, allowing quick, flexible modifications.
3. **Batteries Included:** Python comes with a massive standard library for system tasks, mathematics, text parsing, file handling, and network requests.
4. **C/C++ Extensibility:** Python easily interfaces with compiled lower-level languages. This is crucial because high-performance scientific libraries (like NumPy, TensorFlow, and PyTorch) write their heavy mathematical logic in C/C++ for speed, but expose simple Python interfaces for ease of use.

Major Application Domains

- **Artificial Intelligence & Machine Learning:** Python is the undisputed industry standard for AI. Frameworks like PyTorch, TensorFlow, Scikit-learn, and Keras are built for Python.
 - **Data Science & Scientific Computing:** Powered by NumPy, Pandas, SciPy, and Matplotlib.
 - **Web Development:** Web frameworks like **Django** (batteries-included) and **Flask** (micro-framework) allow for rapid development of web servers and APIs.
 - **Automation & Scripting:** Often used by system administrators to automate repetitive tasks, parse files, and manage cloud infrastructures.
 - **Web Scraping & APIs:** Libraries like BeautifulSoup, Requests, and Scrapy allow developers to harvest large-scale data off the internet.
-

4. Installing Python & Setting Up Your Workspace

To begin programming in Python, you need to install the Python interpreter and choose a development environment that suits your workflow.

Installing Python

1. **Windows:**
 - Download the installer from the official website python.org/downloads.
 - **IMPORTANT:** During installation, check the box that says **"Add Python to PATH"**. If you skip this, your command line will not recognize the **python** command.

2. macOS:

- macOS usually comes with a system version of Python 2.x or 3.x. It is recommended to install the latest Python version using the official installer or via [Homebrew](#):

```
brew install python
```

3. Linux (Ubuntu/Debian):

- Install python via the package manager:

```
sudo apt update  
sudo apt install python3 python3-pip
```

To verify your installation, open your Terminal or Command Prompt and type:

```
python3 --version # Or 'python --version' on Windows
```

5. Introducing Python Development Environments (IDEs)

An Integrated Development Environment (IDE) or text editor is where you write and run your Python code. Here are the four most common choices:

1. IDLE (Integrated Development and Learning Environment)

- **What it is:** Python's built-in, default editor that comes bundled with standard installer packages.
- **Key Features:**
 - Features a simple Interactive Shell (Read-Eval-Print Loop - REPL) where you can type code and see results instantly.
 - Offers basic syntax highlighting and a simple text editor.
- **Best for:** Beginners writing their first scripts or trying out syntax snippets without installing third-party editors.

2. Visual Studio Code (VS Code)

- **What it is:** A free, open-source, lightweight code editor developed by Microsoft.
- **Key Features:**
 - Extensively customizable using extensions (install the **Python** and **Pylance** extensions).
 - Built-in terminal, source control (Git) integration, and highly flexible debugger.
 - Auto-formatting (via **black** or **ruff**) and syntax checking (linting) as you type.
- **Best for:** General-purpose developers, web developers, and system automation engineers who want a fast, extensible editor.

3. PyCharm

- **What it is:** A dedicated Python IDE developed by JetBrains. It comes in a free "Community Edition" and a paid "Professional Edition".
- **Key Features:**
 - Deep code intelligence: advanced autocomplete, automated code refactoring (renaming variables/methods across files), and quick-fix suggestions.
 - Built-in database tools, virtual environment manager, and Django/Flask support (in Professional).
- **Best for:** Large-scale commercial Python projects and developers who want a fully configured, out-of-the-box professional workspace.

4. Jupyter Notebook / JupyterLab

- **What it is:** An open-source web application that allows you to create documents containing live code, equations, visualizations, and narrative text.
- **Key Features:**
 - Code is split into executable "cells" rather than run as a whole script.
 - Remembers variable states in memory between executions, allowing you to run cells out of order.
 - Displays graphs, tables, and HTML directly below the code cells.
- **Best for:** Data Scientists, Machine Learning Engineers, and researchers who perform iterative data explorations and visualization.

6. Python Basic Syntax Guidelines

Before writing programs, you must familiarize yourself with Python's grammar rules. Python syntax is designed to be highly readable, which introduces a few unique rules:

1. Indentation is Mandatory

Unlike C, Java, or C++, which use curly braces `{ }` to define code blocks, Python uses **indentation** (whitespace at the beginning of a line).

- In Python, all statements inside a block (like a loop, function, or conditional) must be indented by the same number of spaces.
- The standard convention is **4 spaces** per indentation level. Do not mix tabs and spaces, as it leads to compilation errors.

2. Line Termination

Python statements are terminated by a **newline** (pressing Enter). Semicolons `;` at the end of a line are **not required** and are generally discouraged.

- If you have a very long statement that you want to split across multiple lines, you can use the backslash line continuation character `\`:

```
total_sum = 1 + 2 + 3 + \  
            4 + 5 + 6
```

3. Case Sensitivity

Python is strictly **case-sensitive**. This means variables named `age`, `Age`, and `AGE` are treated as three completely different, independent variables.

4. Comments

Comments are annotations written in the code to explain what it does. The Python interpreter completely ignores comments during execution.

- **Single-line Comments:** Start with a hash symbol `#`.

```
# This is a single-line comment  
x = 10 # This is an inline comment
```

- **Multi-line Comments / Docstrings:** Written using triple quotes `'''` or `"""`.

```
"""  
This is a multi-line comment  
or docstring, which is often used  
to document functions and classes.  
"""
```

7. The "Hello, World!" Program

The traditional entry point into learning any programming language is printing `"Hello, World!"` to the screen.

Code Implementation

Create a text file named `hello_world.py` and write the following single line:

```
print("Hello, World!")
```

Running the Program

You can run this program in two ways:

Option A: Running as a Script

Open your Terminal or Command Prompt, navigate to the directory where you saved `hello_world.py`, and run:

```
python3 hello_world.py
```

Output:

```
Hello, World!
```

Option B: Running in the Interactive REPL Shell

Open your Terminal, type `python3` (or `python` on Windows) to launch the interactive shell, and type the statement directly:

```
>>> print("Hello, World!")  
Hello, World!
```

To exit the interactive shell, type `exit()` and press Enter.

Anatomy of the Code

- `print()`: This is a built-in Python function that outputs text to the console.
- `"Hello, World!"`: This is a literal string (a sequence of characters). It must be enclosed in double quotes `"..."` or single quotes `'...'` so Python knows it is text and not variable names.

8. Data Types in Python: Scalar vs. Collection Types

Data types determine what kind of value a variable can store and what operations can be performed on it. In Python, data types are broadly divided into two categories: **Scalar (Primitive) Types** and **Collection (Compound) Types**.

A. Scalar Data Types (Single-Value Types)

Scalar data types represent a single value. They are the most basic building blocks in Python.

Data Type	Keyword	Description	Example
Integer	<code>int</code>	Whole numbers, positive or negative, of arbitrary length.	<code>x = -45</code>
Floating-Point	<code>float</code>	Fractional numbers containing decimal points. Supports scientific notations.	<code>y = 3.1415, z = 2.5e3</code>
Complex	<code>complex</code>	Numbers containing a real and an imaginary part (written with a <code>j</code>).	<code>val = 2 + 3j</code>

Data Type	Keyword	Description	Example
Boolean	<code>bool</code>	Represents logical states. Can only be <code>True</code> or <code>False</code> .	<code>is_valid = True</code>
None Type	<code>NoneType</code>	A special constant (<code>None</code>) representing the absence of a value.	<code>data = None</code>

Code Examples for Scalar Types:

```
# Numeric checks
a = 10
b = 3.5
c = 1 + 2j

print(type(a)) # <class 'int'>
print(type(b)) # <class 'float'>
print(type(c)) # <class 'complex'>

# Boolean evaluations
is_greater = 10 > 5 # Evaluates to True
print(is_greater)   # True
print(type(is_greater)) # <class 'bool'>
```

B. Collection Data Types (Multi-Value / Compound Types)

Collection data types store multiple items inside a single variable reference. Python has four primary built-in collection types.

1. Lists (`list`)

- **Description:** Ordered, mutable (changeable) sequences of items. Allows duplicate elements.
- **Syntax:** Square brackets `[...]`
- **Example:** `fruits = ["apple", "banana", 10, True]`

2. Tuples (`tuple`)

- **Description:** Ordered, **immutable** (cannot be modified after creation) sequences of items. Allows duplicate elements.
- **Syntax:** Parentheses `(...)`
- **Example:** `coordinates = (12.97, 77.59)`

3. Dictionaries (`dict`)

- **Description:** Unordered, mutable mappings of key-value pairs. Keys must be unique and immutable.
- **Syntax:** Curly braces with colons `{key: value}`

- **Example:** `student = {"name": "Arham", "age": 24}`

4. Sets (**set**)

- **Description:** Unordered, mutable collections of **unique** elements. Does not allow duplicates.
- **Syntax:** Curly braces `{...}`
- **Example:** `unique_ids = {101, 102, 103, 101}` # Automatically filters duplicate 101

Code Examples for Collection Types:

```
# List vs. Tuple mutability demo
my_list = [1, 2, 3]
my_list[0] = 99 # Valid! my_list becomes [99, 2, 3]

my_tuple = (1, 2, 3)
# my_tuple[0] = 99 # TypeError: 'tuple' object does not support item
assignment

# Dictionary lookup
phone_book = {"Police": 100, "Ambulance": 102}
print(phone_book["Police"]) # 100
```

9. Creating & Using Variables in Python

In Python, a **variable** is a named reference (or label) pointing to an object stored in the computer's memory.

1. Variables are References

When you write `x = 10`, Python does the following:

1. Creates an integer object in memory containing the value `10`.
2. Binds the name `x` to point to that object.
3. If you later reassign `x = "hello"`, Python creates a string object `"hello"`, redirects `x` to point to it, and the old integer `10` is eventually cleaned up by Python's **garbage collector** if nothing else points to it.

```
x = 5
y = x # y now points to the same object as x
print(id(x) == id(y)) # True (they share the same memory location)
```

2. Variable Naming Rules

When naming variables, you must follow these rules:

- Variable names must start with a **letter** or an **underscore** (`_`). They cannot start with a digit.
- They can contain letters, numbers, and underscores (`a-z`, `A-Z`, `0-9`, `_`).
- They cannot contain spaces, punctuation marks, or mathematical symbols.
- They cannot be one of Python's **reserved keywords** (e.g., `if`, `else`, `for`, `while`, `def`, `class`, `import`, `return`, `True`, `False`, `None`).
- Follow standard Python styling conventions (**PEP 8**): Use `snake_case` for variable and function names (e.g., `user_age`, `total_price`).

3. Multiple Assignments

Python allows you to bind multiple variables in a single line:

```
# Bind multiple variables to the same value
x = y = z = 100

# Parallel assignment (unpacking)
name, age, is_student = "Alice", 21, True
```

10. Operators in Python

Operators are special symbols used to perform computations on variables and values.

A. Arithmetic Operators

Used to perform standard mathematical operations:

Operator	Name	Description	Example
<code>+</code>	Addition	Adds two values.	<code>5 + 3 → 8</code>
<code>-</code>	Subtraction	Subtracts second value from first.	<code>5 - 3 → 2</code>
<code>*</code>	Multiplication	Multiplies two values.	<code>5 * 3 → 15</code>
<code>/</code>	Division	Divides and returns a floating-point result.	<code>5 / 2 → 2.5</code>
<code>//</code>	Floor Division	Divides and discards the decimal fraction (truncates down).	<code>5 // 2 → 2</code>
<code>%</code>	Modulo	Returns the division remainder.	<code>5 % 2 → 1</code>
<code>**</code>	Exponentiation	Raises base to the power of exponent.	<code>2 ** 3 → 8</code>

The Difference between `/` and `//`

```
print(10 / 3)    # 3.3333333333333335 (float division)
print(10 // 3)   # 3 (truncates the decimal part, returns int)
```

```
print(-10 // 3) # -4 (rounds down towards negative infinity)
```

B. Comparison (Relational) Operators

Used to compare two values. They always return a Boolean: **True** or **False**.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	True
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>5 > 3</code>	True
<code><</code>	Less than	<code>3 < 5</code>	True
<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	True
<code><=</code>	Less than or equal to	<code>3 <= 5</code>	True

C. Logical Operators

Used to combine conditional statements:

- **and**: Returns **True** if **both** statements are true (e.g., `5 > 3 and 10 > 2` is **True**).
- **or**: Returns **True** if **at least one** statement is true (e.g., `5 > 10 or 10 > 2` is **True**).
- **not**: Reverses the logical state (e.g., `not(5 > 3)` is **False**).

Short-circuit Evaluation:

Logical operators in Python use short-circuiting:

- In `A and B`, if `A` is **False**, Python does not evaluate `B` because the overall result is guaranteed to be **False**.
 - In `A or B`, if `A` is **True**, Python does not evaluate `B` because the overall result is guaranteed to be **True**.
-

D. Assignment Operators

Used to assign values to variables, often combined with arithmetic operations (shorthand operators):

```
x = 10    # Standard assignment
x += 5    # Equivalent to x = x + 5 (x becomes 15)
x -= 2    # Equivalent to x = x - 2 (x becomes 13)
x *= 2    # Equivalent to x = x * 2 (x becomes 26)
x /= 2    # Equivalent to x = x / 2 (x becomes 13.0)
```

11. Basic Input/Output (I/O) Operations in Python

A program interacts with users by taking data in (Input) and showing results back (Output). In Python, this is primarily managed by the built-in functions `print()` and `input()`.

A. Output Operations: `print()`

The `print()` function writes data to the standard output (usually the terminal screen).

1. Printing Multiple Values

You can print multiple variables or values in a single call by separating them with commas. By default, Python separates them with a space.

```
name = "Alice"
age = 21
print("Name:", name, "Age:", age) # Output: Name: Alice Age: 21
```

2. Custom Separators (`sep=`)

You can override the default space separator between items using the `sep` parameter.

```
print("24", "08", "2026", sep="-") # Output: 24-08-2026
```

3. Custom Line Endings (`end=`)

By default, the `print()` function appends a newline character (`\n`) at the end of the print statement. You can change this using the `end` parameter.

```
print("Hello", end=" ")
print("World") # Output: Hello World (on the same line)
```

4. Formatting Output Strings

There are three main ways to inject variables into output strings:

- **Method 1: String Concatenation** (Legacy / Tedious)
 - Requires manually casting variables to strings.

```
print("Age: " + str(age))
```

- **Method 2: `.format()` method** (Legacy)

```
print("Name: {}, Age: {}".format(name, age))
```

- **Method 3: F-strings (Formatted String Literals)** (Modern / Recommended)

- Prefix the string with an `f` or `F`, and write variables directly inside curly braces `{}`. It is faster, cleaner, and allows evaluating expressions.

```
print(f"Name: {name}, Age: {age}")
print(f"Double of age is: {age * 2}")
```

B. Input Operations: `input()`

The `input()` function pauses program execution and waits for the user to type text on the keyboard and press Enter.

[!IMPORTANT] The `input()` function **ALWAYS** returns the user's input as a string (`str`). If you need numeric values, you must convert (cast) them explicitly using functions like `int()` or `float()`.

Handling String Input:

```
user_name = input("Enter your username: ")
print(f"Hello, {user_name}!")
```

Handling Numeric Input (Casting):

```
# Convert to integer
qty = int(input("Enter quantity: "))
price = float(input("Enter unit price: "))

total_cost = qty * price
print(f"Total Cost: ${total_cost:.2f}") # ':.2f' limits output to 2
decimal places
```

What happens if you forget to cast?

If you try to perform arithmetic operations directly on string inputs, Python will perform string concatenation (for `+`) or raise a `TypeError` (for other operators like `-`, `*`, `/`):

```
x = input("Enter first number: ") # User enters: 5
y = input("Enter second number: ") # User enters: 3
```

```
print(x + y) # Output: "53" (concatenates strings rather than adding
numbers!)
```

12. Flow of Control: Conditional Statements

By default, Python executes statements sequentially from top to bottom. Conditional statements allow you to diverge this execution path by running certain blocks of code only if specific conditions are met.

In Python, conditional flow is controlled by the keywords `if`, `elif` (short for else-if), and `else`.

A. Core Rules of Conditional Statements

1. **The Colon (:)**: Every conditional statement header (`if`, `elif`, `else`) must end with a colon.
 2. **Indentation**: The code block to be executed if a condition is met must be indented (standard 4 spaces). The end of the block is marked by returning to the outer indentation level.
 3. **Condition Expression**: Python evaluates the expression after `if` or `elif` as a Boolean (`True` or `False`).
-

B. The `if` Statement

Runs a block of code only if the condition evaluates to `True`.

```
temperature = 35

if temperature > 30:
    print("It is a hot day!") # Runs only if temperature > 30
print("Drive safely.")      # Always runs (outside the if block)
```

C. The `if-else` Statement

Provides an alternative execution block when the condition is `False`.

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are too young to vote.")
```

D. The `if-elif-else` Chain

Used to check multiple mutually-exclusive conditions in sequence. Python checks the conditions from top to bottom and executes **only the first block** whose condition is **True**. All subsequent blocks are skipped.

```
score = int(input("Enter your exam score (0-100): "))

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

E. Nested **if-else** Statements

You can place conditional structures inside other conditional blocks to resolve complex, dependent criteria.

```
has_license = True
age = 20

if age >= 18:
    print("Age verified.")
    if has_license:
        print("You are allowed to rent a car.")
    else:
        print("You need a valid license to rent a car.")
else:
    print("You are too young to drive.")
```

F. Truthy and Falsy Values in Python

In Python, values of non-Boolean data types can be implicitly evaluated in conditional tests.

- **Falsy Values:** Evaluate to **False** in conditions:
 - **None**
 - **False**
 - **0** (integer zero)
 - **0.0** (float zero)
 - **""** (empty string)
 - **[]** (empty list), **()** (empty tuple), **{}** (empty dictionary/set)
- **Truthy Values:** Any value not on the Falsy list evaluates to **True**.

```
# checking for empty lists or strings pythonically
name = input("Enter name: ")
if name: # Evaluates to True if name is not an empty string
    print(f"Hi {name}!")
else:
    print("You didn't enter a name!")
```

13. Looping Structures in Python

Loops are used to repeatedly execute a block of code. Python supports two main loop types: **while** loops and **for** loops.

A. The **while** Loop

A **while** loop repeatedly executes a block of code as long as a specified condition remains **True**.

```
count = 1
while count <= 5:
    print(f"Count is: {count}")
    count += 1 # IMPORTANT: Update condition variable to prevent an
infinite loop!
```

Infinite Loops

If the loop condition never evaluates to **False**, the loop runs forever, freezing your program.

```
# Warning: Infinite Loop!
# count = 1
# while count <= 5:
#     print(count) # Missing 'count += 1', count stays 1 forever
```

Press **Ctrl + C** in your terminal to force-terminate an infinite loop.

B. The **for** Loop

A **for** loop is used to iterate over a sequence (such as a string, list, tuple, set, dictionary, or a numeric range).

1. Iterating over a string

```
word = "Python"
for letter in word:
```

```
print(letter) # Prints each character on a new line
```

2. The `range()` Function

To run a loop a specific number of times, combine the `for` loop with the built-in `range()` function.

- `range(stop)`: Runs from `0` up to `stop - 1` (stop is exclusive).

```
for i in range(3):  
    print(i) # Prints: 0, 1, 2
```

- `range(start, stop)`: Runs from `start` up to `stop - 1`.

```
for i in range(2, 6):  
    print(i) # Prints: 2, 3, 4, 5
```

- `range(start, stop, step)`: Runs from `start` to `stop - 1`, incrementing by `step` each time.

```
for i in range(1, 10, 2):  
    print(i) # Prints odd numbers: 1, 3, 5, 7, 9
```

C. Loop Control Statements: `break` and `continue`

You can alter the standard execution of a loop using `break` and `continue`.

- **`break`**: Terminates the loop immediately.

```
# Search for value 7  
for num in range(1, 10):  
    if num == 7:  
        print("Found 7! Stopping search.")  
        break  
    print(f"Checking {num}...")
```

- **`continue`**: Skips the rest of the current iteration and jumps directly to the next cycle.

```
# Print numbers 1-5 except 3
for num in range(1, 6):
    if num == 3:
        continue # Skip the print line below for 3
    print(num)
```

D. The Unique `else` Clause in Loops

In Python, loops can have an optional `else` block.

- **Rule:** The code in the `else` block runs **only if the loop finishes successfully without encountering a `break` statement.**
- **Use Case:** Ideal for search operations to run "not found" fallback code.

```
# Search for even numbers in a list
numbers = [1, 3, 5, 7]

for num in numbers:
    if num % 2 == 0:
        print(f"Found even number: {num}")
        break
else:
    # Runs only if the loop finished without hitting 'break'
    print("No even numbers found in the list.")
```

E. Nested Loops

A loop written inside the body of another loop.

```
# Print coordinates grid
for x in range(1, 3):
    for y in range(1, 4):
        print(f"({x}, {y})", end=" ")
    print() # Line break after inner loop completes
(1, 1) (1, 2) (1, 3)
(2, 1) (2, 2) (2, 3)
```

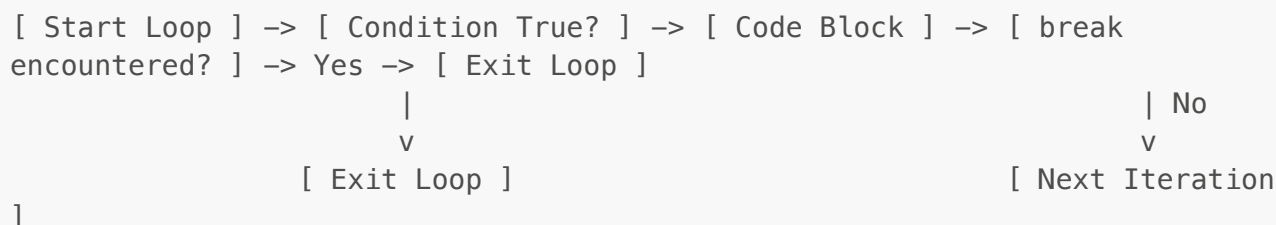
14. Loop Control Structures: `break`, `continue`, and `pass`

While writing loops, you often need to alter the flow of iteration based on external conditions. Python provides three core control keywords: `break`, `continue`, and `pass`.

A. The `break` Statement

The `break` statement immediately terminates the current loop execution. Program control jumps directly to the first statement outside the loop block.

Flow Diagram Analogy:



Practical Example:

A simple console menu that loops indefinitely until the user chooses to exit:

```

while True:
    choice = input("Enter 'q' to quit, any other key to continue: ")
    if choice.lower() == 'q':
        print("Exiting loop...")
        break # Immediately exits the while loop
    print("Running process...")
    print("Program continues here.")

```

B. The `continue` Statement

The `continue` statement skips the remaining code statements inside the loop body for the **current iteration only**, and immediately jumps to the next cycle of the loop (re-evaluates the loop condition).

Practical Example:

Printing only odd numbers from a list:

```

numbers = [1, 2, 3, 4, 5, 6]

for num in numbers:
    if num % 2 == 0:
        continue # Skips print(num) for even numbers and goes to next loop
    iteration
    print(f"Odd number: {num}")

```

Output:

```
Odd number: 1
Odd number: 3
Odd number: 5
```

C. The `pass` Statement

The `pass` statement is a **null operation**—nothing happens when it executes.

- **Why do we need it?:** Python relies on indentation blocks. If you write a loop, function, or class block with no body, Python will crash with an `IndentationError`. The `pass` statement serves as a syntactic placeholder.

Practical Example:

```
# 1. Placeholder in a loop to write logic later
for i in range(100):
    pass # Keeps the loop valid without raising IndentationError

# 2. Placeholder in a conditional branch
if score > 90:
    pass # TODO: Add bonus marks logic
else:
    print("No change.")

# 3. Placeholder for empty function shells
def fetch_api_data():
    pass # Skeleton definition
```

D. Side-by-Side Comparison

Feature	<code>break</code>	<code>continue</code>	<code>pass</code>
Action	Terminates the loop structure immediately.	Skips current loop cycle and starts the next.	Does nothing; acts as a syntactic placeholder.
Loop Exit?	Yes	No	No
Line Skip?	Yes (all remaining lines and iterations)	Yes (remaining lines of current cycle only)	No (all lines continue executing normally)
Syntax Role	Behavioral control.	Behavioral control.	Syntactic placeholder only.